



TG1: Trabalho em Grupo sobre Python

Sistema de Votação

Objetivo

Durante a disciplina de programação orientada a dados, vocês estão aprendendo a programar na linguagem de programação Python, que é multi-paradigma. Neste trabalho, o objetivo é explorar especificamente conceitos relacionados ao paradigma de Programação Orientada a Objetos (POO), módulos e manipulação de arquivos.

Contexto

A temática do trabalho é desenvolver um sistema simplificado de eleições municipais, utilizando os conceitos vistos em aula. Nas eleições municipais, os eleitores votam para os cargos de prefeito e vereador, sendo o prefeito eleito responsável pelo Poder Executivo municipal, enquanto o grupo de vereadores eleitos atua no Poder Legislativo. Nesse contexto, você é um programador responsável por desenvolver um sistema capaz de gerenciar uma instância de eleição, considerando: eleitores, cargos (prefeito e vereador), partidos e urnas.

Restrições

Os conceitos explorados neste trabalho são fundamentais para o desenvolvimento profissional de um cientista de dados. Portanto, para fins de avaliação do aprendizado, não é permitido o uso de inteligência artificial no desenvolvimento.

É possível implementar este trabalho importando apenas os módulos `'abc'` e `'sys'` do Python. Entretanto, podem ser usados quaisquer outros pacotes vistos em aula. Pacotes e módulos não vistos em aula não são permitidos no trabalho.

Especificações

O trabalho deve ser organizado como um pacote do Python que contém as classes e métodos que irão coordenar as eleições. Também deve conter uma aplicação que usa esse pacote para realizar o processamento dos arquivos de entrada e simular a eleição. O trabalho tem as seguintes especificações:

1. Todo o sistema deve ser implementado como um pacote do Python;
2. A organização interna do pacote fica a critério dos alunos. O pacote pode conter um ou mais módulos com todas as definições das classes, ou diferentes módulos separados;
3. Deve haver uma classe **'Partido'** com os seguintes atributos:
 - Número;
 - Sigla;
 - Nome (deve ter no máximo 50 caracteres);
 - CNPJ (deve ter exatamente 14 caracteres numéricos).

4. A classe '**Partido**' deve ter um atributo de classe para contabilizar o número de ocorrências de erros de validação de 'nome' ou 'CNPJ' ao tentar criar uma instância de partido. Esse atributo de classe deve ser incrementado por meio de um método de classe.
5. Deve haver uma classe '**Eleitor**' com os seguintes atributos:
 - Nome (deve ter no máximo 50 caracteres);
 - CPF (deve ter exatamente 11 caracteres numéricos);
 - Data de nascimento;
 - Título de eleitor;
 - Zona;
 - Seção.
6. Considerando que candidatos também votam em uma eleição, deve haver uma subclasse '**Candidato**', que estende a classe '**Eleitor**' com os seguintes atributos adicionais:
 - Número (representa o candidato na votação);
 - Partido (recebe o objeto do partido correspondente do candidato).
7. Essa classe '**Candidato**' deve ser abstrata e conter a assinatura de um método para verificação/validação do número do candidato (conforme descrito no item 10). Esse método estabelece um contrato com as subclasses de candidato para que implementem esse método;
8. Deve haver uma subclasse '**Prefeito**', que estende a classe '**Candidato**' com os seguintes atributos adicionais:
 - CPF do vice-prefeito (deve ter exatamente 11 caracteres numéricos);
 - Plano de governo.
9. Deve haver uma subclasse '**Vereador**', que estende a classe '**Candidato**' com os seguintes atributos adicionais:
 - Bairro;
 - Projeto de lei.
10. As classes '**Prefeito**' e '**Vereador**' devem implementar o método de verificação definido na classe abstrata '**Candidato**'. Para prefeito, o atributo 'número' deve ter exatamente 2 dígitos, enquanto para vereador, o atributo 'número' deve ter exatamente 5 dígitos.
11. As classes '**Prefeito**' e '**Vereador**' devem ter um atributo de classe para contabilizar o número de vezes em que houve erros de validação no número do candidato ao tentar criar uma instância da respectiva classe. Esses atributos de classe devem ser incrementados a partir de um método de classe.
12. Deve haver uma classe '**Urna**' com os seguintes atributos:
 - Zona;
 - Seção;
 - Votos para prefeito (mapeia a quantidade de votos de cada prefeito);
 - Votos para vereador (mapeia a quantidade de votos de cada vereador).
13. A classe '**Urna**' deve implementar o conceito de sobrecarga de operadores. Como a eleição tem várias urnas, deverá ser possível somar dois objetos do tipo urna. A instância resultante deve ter os votos agregados e os atributos de zona e seção configurados para '**None**';
14. Deve haver uma classe '**Eleicao**' que gerenciará a eleição por meio de listas e de métodos de inserção e busca de objetos nelas. Essa classe deve ser capaz de gerar relatórios referentes aos eleitores, partidos, urnas e resultados;
15. Deve haver uma classe personalizada de exceção (ex. '**ErroCadastro**'). Ela deve ser utilizada sempre que alguma validação exigida na criação de qualquer instância definida no trabalho falhar. A mensagem da exceção deve informar o tipo do objeto, o campo que gerou o erro e o respectivo valor. Quando uma linha for inválida, ela deve ser ignorada, e o

- processamento deve continuar.
16. Deve ser implementado o controle de acesso aos dados por meio de decoradores ou descritores. O tratamento de erros e exceções precisa ser implementado conforme as boas práticas vistas em aula.
 17. A aplicação que simula a eleição usando o pacote criado deve receber como argumento na linha de comando os arquivos de entrada seguindo o padrão do exemplo abaixo:
 - `python3 app.py Partidos.txt Eleitores.txt Urnas.txt`
 18. O formato dos dados de entrada está descrito no topo dos respectivos arquivos disponibilizados no Moodle.
 19. Após a leitura dos dados, devem ser gerados 5 relatórios sobre os seguintes aspectos da eleição: eleitores, partidos, urnas, resultado geral e erros. A geração dos relatórios deve ser implementada como métodos da classe **'Eleicao'** e chamada pela aplicação. Para cada relatório devem ser informados os seguintes dados:
 - Sobre os eleitores:
 - Total de eleitores (candidatos também são eleitores);
 - Total de candidatos a prefeito;
 - Total de candidatos a vereador;
 - Para cada Zona, informar as Seções existentes e o respectivo número de eleitores por Seção.
 - Sobre os partidos:
 - Total de partidos;
 - Para cada partido, informar o nome, a sigla e o número de candidatos a prefeito e vereador.
 - Sobre as urnas:
 - Relatório individual de cada urna, identificada pela zona e seção, contendo o número de votos válidos, brancos e nulos para prefeito e o mesmo para vereador. Votos devem ser considerados nulos quando o valor informado não corresponder a nenhum candidato existente e brancos quando não for informado nenhum valor no campo.
 - Sobre o resultado geral:
 - Informar o nome do prefeito ganhador (se nenhum prefeito tiver obtido pelo menos 50% dos votos válidos, deve informar: 'Segundo Turno');
 - Informar o nome dos 10 vereadores com a maior quantidade de votos e o seu respectivo número de votos recebidos.
 - Sobre os erros e exceções:
 - Informar cada mensagem gerada pelas classes de erro;
 - Informar os valores dos atributos que contabilizam os erros nas classes **'Partido'**, **'Prefeito'** e **'Vereador'**.
 20. Durante o desenvolvimento, é recomendado usar a tipagem do Python (Type Hints) para facilitar o entendimento do código e identificação de problemas.

Relato

Faça um resumo de no máximo 400 palavras relatando como o trabalho foi dividido entre o grupo, as dificuldades e outras observações que achar relevantes para comentar sobre a realização deste trabalho. Você também pode comentar sobre sua evolução no conhecimento. Esse resumo deve ser enviado junto com os demais arquivos.

Entrega

Você deve enviar um pull request contendo o seguinte:

- eleicao (diretório do pacote com o(s) módulo(s) criado(s));
- Input (diretório onde devem estar os arquivos de entrada);
- output (diretório onde devem ser gerados os arquivos de relatório);
- app.py (aplicativo que usa o pacote 'eleicao' e simula o sistema);
- README.md (arquivo descrevendo como usar o aplicativo e o pacote);
- RELATO.md (arquivo contendo o relato de 400 palavras).